POC

Goal

In this document, we are formalizing all the informations concerning the POC done between our mobile and IOT devices.

The goal of the POC is to measure the precision possible to get with our IOT devices, for us an ESP-32. The “precision” is typically if my system locate me here, how far I am? Is that centimeters? meters?

The goal is also to have a smaller representation of the reality, the scale should not be a problem.

Material used

- Raspberry PI4
- ESP-32
- Personal phone

Steps to follow:

1. Enable SSH from the Raspberry PI4.
2. Establish a connection between the ESP-32 and the Raspberry PI4.
3. Establish a connection between a personal phone and an ESP-32. The ESP-32 will collect information of the phone such as its MAC address and its RSSI.
4. Send the mobile phone data from the ESP-32 to the Raspberry.
5. With all the data sent, the Raspberry will have all the information at its disposal to approximately locate the user. This step is the calculation of the distance between the phone and the ESP32 (in order to locate the user).
6. Use several ESP32s in order to be able to triangulate the distance between each of them.

Duration

The POC must be completed within 9 days.

Annex

Script ESP 32

(BELOW) Code to scan via BLE for devices and return in the monitor the RSSI / @mac / device name (if available).

```
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEScan.h>
#include <BLEAdvertisedDevice.h>

int scanTime = 3; // durée du scan en secondes
BLEScan* pBLEScan;

void setup() {
  Serial.begin(115200);
  Serial.println("Démarrage du scan BLE...");

```

```
  BLEDevice::init("");
  pBLEScan = BLEDevice::getScan(); // créer un nouveau scan
  pBLEScan->setActiveScan(true); // le scan actif utilise plus de puissance, mais obtient des résultats plus rap
  pBLEScan->setInterval(100);
  pBLEScan->setWindow(99);  // valeur inférieure ou égale à setInterval
}

void loop() {
  // démarrer le scan
  BLEScanResults foundDevices = pBLEScan->start(scanTime, false);

  // afficher le nombre de périphériques trouvés
  Serial.print("Périphériques trouvés : ");
  Serial.println(foundDevices.getCount());

  // afficher les informations de chaque périphérique trouvé
  for (int i = 0; i < foundDevices.getCount(); i++) {
    BLEAdvertisedDevice device = foundDevices.getDevice(i);
    Serial.print("Adresse du périphérique : ");
    Serial.println(device.getAddress().toString().c_str());
    Serial.print("Nom de l'appareil : ");
    Serial.println(device.getName().c_str());
    Serial.print("RSSI : ");
    Serial.println(device.getRSSI());
  }

  // effacer les résultats du buffer de scan pour libérer de la mémoire
  pBLEScan->clearResults();
}


```

(BELOW) Code to make an ESP32 a wifi access point.

```
#include <WiFi.h>

const char* ssid = "ESP32-Access-Point";
const char* password = "123456789";

WiFiServer server(80);

void setup() {
  Serial.begin(115200);

  WiFi.softAP(ssid, password);

  IPAddress IP = WiFi.softAPIP();
  Serial.print("AP IP address: ");
  Serial.println(IP);

  server.begin();
}

void loop() {
  WiFiClient client = server.available();   // listen for incoming clients

  if (client) {                             // if you get a client,
    Serial.println("New Client.");          // print a message out the serial port
    String currentLine = "";                // make a String to hold incoming data from the client
```

```
    while (client.connected()) {            // loop while the client's connected
      if (client.available()) {             // if there's bytes to read from the client,
        char c = client.read();             // read a byte, then
        Serial.write(c);                    // print it out the serial monitor
        if (c == '\\n') {                    // if the byte is a newline character

          // if the current line is blank, you got two newline characters in a row.
          // that's the end of the client HTTP request, so send a response:
          if (currentLine.length() == 0) {
            // HTTP headers always start with a response code (e.g. HTTP/1.1 200 OK)
            // and a content-type so the client knows what's coming, then a blank line:
            client.println("HTTP/1.1 200 OK");
            client.println("Content-type:text/html");
            client.println("Connection: close");
            client.println();

            // Display the RSSI for each client
            int rssi = WiFi.RSSI(client.remoteIP());
            client.print("RSSI: ");
            client.println(rssi);

            // The HTTP response ends with another blank line:
            client.println();
            // break out of the while loop:
            break;
          } else {    // if you got a newline, then clear currentLine:
            currentLine = "";
          }
        } else if (c != '\\r') {  // if you got anything else but a carriage return character,
          currentLine += c;      // add it to the end of the currentLine
        }
      }
    }
    // close the connection:
    client.stop();
    Serial.println("Client Disconnected.");
  }
}


```

(BELOW) Code to create a BLE access point and send "hello from ESP32!".

```
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

BLEServer* pServer = NULL;
BLECharacteristic* pCharacteristic = NULL;
bool deviceConnected = false;

// UUIDs pour le service et la caractéristique
#define SERVICE_UUID        "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHARACTERISTIC_UUID "beb5483e-36e1-4688-b7f5-ea07361b26a8"

class MyServerCallbacks: public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) {
      deviceConnected = true;
```

```
    };

    void onDisconnect(BLEServer* pServer) {
      deviceConnected = false;
    }
};

void setup() {
  Serial.begin(115200);

  // Créer le périphérique BLE
  BLEDevice::init("ESP32 BLE");

  // Créer le serveur BLE
  pServer = BLEDevice::createServer();
  pServer->setCallbacks(new MyServerCallbacks());

  // Créer le service BLE
  BLEService *pService = pServer->createService(SERVICE_UUID);

  // Créer la caractéristique BLE
  pCharacteristic = pService->createCharacteristic(
                      CHARACTERISTIC_UUID,
                      BLECharacteristic::PROPERTY_READ   |
                      BLECharacteristic::PROPERTY_WRITE  |
                      BLECharacteristic::PROPERTY_NOTIFY |
                      BLECharacteristic::PROPERTY_INDICATE
                    );

  // Démarrer le service
  pService->start();

  // Démarrer l'annonce
  pServer->getAdvertising()->start();
  Serial.println("En attente d'une connexion...");
}

void loop() {
  // Mettre à jour la caractéristique si un périphérique est connecté
  if (deviceConnected) {
    String value = "Hello from ESP32!";
    pCharacteristic->setValue(value.c_str());
    pCharacteristic->notify();
    delay(1000);
  }
}


```

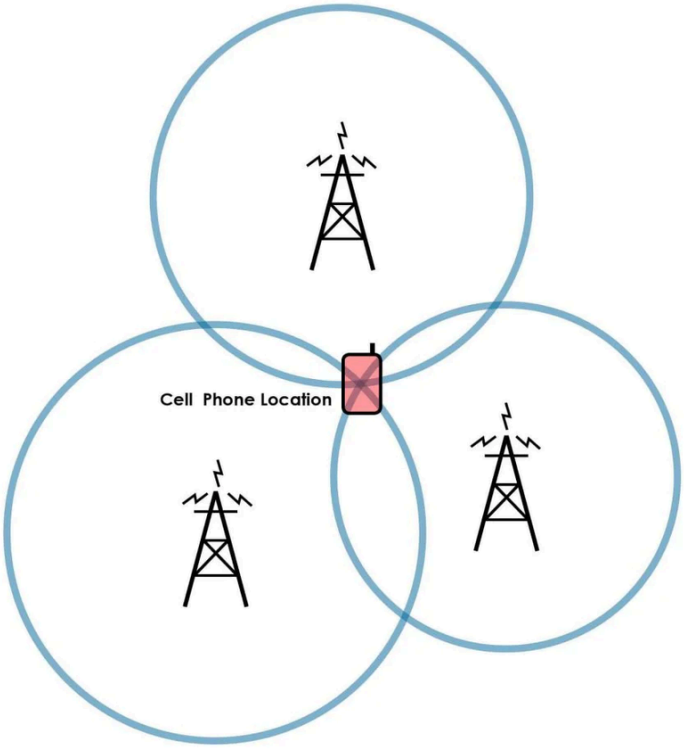
(BELOW) Code that retrieves peripheral RSSI via BLE and connects to an access point through wifi.

```
#include <NimBLEDevice.h>
#include <NimBLEScan.h>
#include <NimBLEAdvertisedDevice.h>
#include <WiFi.h>

const char* ssid = ""; //remplacer par le SSID du PA
const char* password = ""; //remplacer par le MDP du PA
const char* targetMAC = "11:22:33:44:55:66";  // Remplacez par l'adresse MAC que vous recherchez
```

```

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(15000);
  }

  Serial.println("Connecté au WiFi");
  NimBLEDevice::init("");
  Serial.println("BLE Initialiser");
}

void loop() {

  if (WiFi.status() != WL_CONNECTED) {
    WiFi.begin(ssid, password);
    delay(15000);
  }
  Serial.println("Boucle loop début!");
  NimBLEScan* pNimBLEScan = NimBLEDevice::getScan();
  pNimBLEScan->setActiveScan(true);
  NimBLEScanResults foundDevices = pNimBLEScan->start(5);

  for (int i = 0; i < foundDevices.getCount(); i++) {
    NimBLEAdvertisedDevice advertisedDevice = foundDevices.getDevice(i);
    Serial.printf("Adresse MAC: %s, RSSI: %d, Nom: %s\\n", advertisedDevice.getAddress().toString().c_str(), adv
  }
  delay(5000);  //temps en millisecondes.
}


```

Triangulation

The triangulation involves measuring the angles from two known points to the target point, as well as the distance between the known points. With this information, the distance to the target point and its relative position can be calculated using trigonometric equations.



Links :

UDP ESP to RasBerryPI4 : Communication UDP entre Raspberry Pi et ESP32 • AranaCorp

Repository of **localization :** GitHub - KMuthumala/ESP32-BLE-Beaconing: Determining indoor locations using bluetooth beaconing approach

Script calcul RSSI to range

```
def calcul_range(RSSI, A, n):
    """
    Calculates distance from RSSI using formula:
    d = 10^((ABS(RSSI) - A) / (10 * n))
    Sources : https://projets-ima.plil.fr/mediawiki/index.php/Mesure_de_distance_par_RSSI#:~:text=D%27abord%2C%2

    Args:
    -RSSI (float): The value of the Received Signal Strength Indication (RSSI).
    -A (float): The absolute value of the power at a distance of one meter from the transmitter.
    -n (float): The path loss exponent.

    Returns:
    - float: La distance calculée en mètres.
    """
    range= 10 ** ((abs(RSSI) - A) / (10 * n))
    return range

# Params
```

```
RSSI = -55
A = 43.66
# A = 50
n = 2.2

# Calculrange
range= calcul_range(RSSI, A, n)
print(f"range: {range} m")
```

Result of POC

Note: What we notice is that if we're in an environment with a lot of Bluetooth devices, the accuracy with RSSI is incorrect. The more devices there are, the less viable the accuracy calculated via RSSI becomes.
~~Il y a aussi un changement en fonction de la distance a partir de ….m le RSSI n'est plus du tout précis~~

Test with one ESP32

For these tests we tested in 2 different environments :

1. Lot of Bluetooth device
2. Low device Bluetooth devide

Environnement	Distance	RSSI	Distance calculé
Lot of Bluetooth device in a barrier-free room divice>100	50 cm	-46	65cm
Lot of Bluetooth device in a barrier-free room divice>100	1 m	-55	1.68 m
Lot of Bluetooth device in a barrier-free room divice>100	5m	-65	4.80 m
Low device Bluetooth devide	50 cm		
Low device Bluetooth devide	1 m		
Low device Bluetooth devide	5m		

Conclusion

The margin of error for position calculation using BLE technology with an esp32 map is greater than our tolerance, making it impossible to correctly validate the position of a person on a map. We need to couple this localization method with a second method that will improve the position and therefore fall within our error tolerance.

We have considered the following solutions to improve our accuracy:

Solution 1 : We thought of using UWB technology, which offers greater accuracy, but this is not the case.

Ultra-Wideband (UWB) is a short-range wireless communication protocol that operates over radio waves with a wider frequency range than BLE, enabling reliable and secure telemetry and precision detection, creating a new dimension of spatial context for wireless devices.

Devices start to “telemeter”. Telemetry involves calculating the time of flight (ToF) between devices: the round-trip time of request/response packets, unlike BLE which is calculated by signal strength (RSSI).

UWB TOF systems are used in a variety of applications, including :

- Indoor Positioning Systems (IPS): For locating people and goods inside buildings.
- Automotive Safety: For collision avoidance and parking assistance systems.
- Healthcare: For tracking patients and medical equipment.
- Consumer Electronics: In smart devices for gesture recognition and spatial awareness.

One on the point is the frequencies is diferent of Wi-Fi and BLE which helps avoid interference.

	BLE	Wi-Fi	UWB	Chirp (CSS)
Location Accuracy*	< 5 m	< 10 m	10-50 cm	1-2 m
Range*	Optimal: 0-25 m Up to 100 m	Optimal: 0-50 m Up to 500 m	Optimal: 0-50 m Up to 200 m	Optimal: 10-500 m Up to 1000 m
Latency*	Typically 3-5 s to get location	Typically 3-5 s to get location	< 1 ms to get location	< 1 ms to get location
Power Consumption	Very low, option for embedded cell battery in select hardware options	Moderate	Low, option for embedded cell battery in select hardware options	Very low, option for embedded cell battery in select hardware options
Cost	$$	$$$ (Low $ with existing Wi-Fi access points)	$$	$
Frequencies	2.4 GHz	2.4, 5 GHz	3.1 – 10.6 GHz	ISM-band 2.4 GHz (2.4-2.4835)
Data Rate	Up to 2 Mbps	Up to 1 GBps	Up to 27 Mbps	Up to 2 Mbps

*Sources: (link, link2, link3)

- **Solution 2 :**
  - With the ESP32 (classic) you can use addition to or in combination with the built-in Wi-Fi of the ESP32Wi-Fi RTT (Round Trip Time). Wi-Fi RTT technology, which allows measuring the time of flight of Wi-Fi signals to estimate the distance between the access point and the device. This technology can offer location accuracy up to one meter.
  - **Advantages:**
    - Reasonable accuracy: Better than RSSI.
    - Compatibility: Uses existing Wi-Fi hardware.
  - **Disadvantages:**
    - Requires RTT-compatible access points.
    - Sensitivity to obstacles and interference.

*Sources:(link)

- **Solution 3 :**

- Odometry is a method used to estimate the change in position of a mobile robot. It is often used in robotics to track a robot's trajectory. We would use this other type of technology to complete the position calculation with BLE or wifi.
- **Advantages:**
  - Simplicity: Odometry is a simple and straightforward method of estimating a robot's position1.
    Independence: It is not dependent on the external environment, making it useful in environments where other location methods may be imprecise or unavailable.
  - Speed: Odometry can provide real-time position estimates, which is crucial for many robotics applications.
- **Disadvantages** :
  - Cumulative error: Error in position estimation accumulates over time, which can lead to increasingly inaccurate estimates.
  - Sensitivity to surface conditions: Odometry is sensitive to the conditions of the surface on which the robot is moving. For example, a slippery or uneven surface can lead to estimation errors.
  - Limited to motion detection: Odometry can only detect robot movement and cannot provide information about the surrounding environment.

*Sources:(link, link2)